Informe. Comparar la implementación de un diccionario de datos físico en diferentes SGBD

Integrantes del equipo:

Karen Julieth Castro Arias

Jorge Eliecer Mazo Uparela

Samuel Hincapié Pérez

Paula Andrea Morales Zapata

Ali Jose Arias Basto

Curso: Bases de Datos I – G100 (SD1006)

Oralia Cortes Grajales

Enlace al repositorio de GitHub:

https://github.com/Jorge-Mazo/bd1-redsocial-grupo6

Enlace al video:

https://drive.google.com/file/d/1_zvJPvTaFzFHtXz9ScGFbPVTTYBCkhsu/view

25 de octubre de 2026

Introducción

En la Tarea 3 implementamos el modelo físico de la red social en PostgreSQL 16. En esta tarea lo llevamos a tres SGBD distintos (PostgreSQL 16, MySQL 8.0 y MariaDB 10.11) para comparar cómo cada motor resuelve lo mismo: tipos de datos, llaves, restricciones, comentarios y datos semiestructurados.

Además, agregamos al modelo un campo JSON, metadatos_dispositivo, en la tabla publicacion, y lo justificamos con un caso de uso de big data e IoT.

Los tres scripts crean las mismas 24 tablas y 33 llaves foráneas. Los ejecutamos en motores reales y verificamos el resultado con consultas al catálogo de cada uno. El informe incluye la justificación de los motores, la implementación paso a paso, la comparación de tipos, las dificultades, las conclusiones y las propuestas de mejora.

Justificación de los tres SGBD

Elegimos tres motores gratuitos que todo el equipo podía instalar y probar sin costo.

PostgreSQL 16. Ya era nuestro motor de la Tarea 3. Cumple muy bien el estándar SQL y tiene jsonb con índices GIN, timestamptz, boolean real y COMMENT ON. Sirve de referencia contra la que medimos a los otros dos.

MySQL 8.0. Es uno de los motores más usados en aplicaciones web. Es ligero, tiene tipo JSON nativo con validación y herramienta gráfica gratuita (MySQL Workbench). Permite ver qué se pierde o se gana al pasar de PostgreSQL a un motor más simple.

MariaDB 10.11. Nació como bifurcación de MySQL y mantiene compatibilidad con su sintaxis. Por eso es una comparación interesante: son dos motores “parecidos” que, como veremos, difieren en cómo guardan el JSON.

Descartamos SQL Server y Oracle por licenciamiento y requisitos de instalación

Criterio	PostgreSQL 16	MySQL 8.0	MariaDB 10.11
Licencia	Código abierto	Gratuito (edición Community)	Código abierto
Tipo JSON	jsonb (binario)	json (binario)	json es alias de longtext
Fecha con zona horaria	timestamptz	No existe	No existe
Índice sobre JSON	GIN	Índice funcional	Columna virtual indexada
Herramienta gráfica	pgAdmin 4	MySQL Workbench	HeidiSQL / DBeaver

Diccionario de datos genérico actualizado

Partimos del diccionario de la Tarea 2 (sección 8) y lo actualizamos con los cambios que hicimos al pasar a la implementación física:

1. **Campo nuevo.** PUBLICACION.metadatos_dispositivo, tipo genérico “JSON / semiestructurado”, opcional.
2. **Fechas con hora.** Las fechas de interacción (publicación, comentario, mensaje, entre otras) pasaron de “fecha” a “fecha y hora”, porque un muro social necesita orden cronológico. En el diccionario original MENSAJE.fecha_envio decía “fecha y hora” pero tenía tipo DATE; lo corregimos.
3. **Llave foránea compuesta.** En SESION_MENTORIA, el par (mentor, materia) ahora referencia a MENTOR_MATERIA_HABILITADA. Así solo se puede registrar una sesión con un mentor habilitado para esa materia.
4. **Restricciones nuevas.** Se incluyeron las restricciones CHECK definidas en la Tarea 3.

   El diccionario genérico completo se entrega en la carpeta diccionarios/.

Implementación paso a paso

Pasos comunes a los tres motores:

Del modelo lógico al modelo físico

El modelo lógico define entidades, atributos, llaves y cardinalidades sin depender de ningún sistema. En el físico las decisiones son concretas, y esto fue lo que hicimos con cada elemento:

- Cada entidad se convirtió en una tabla (24 en total) y cada atributo en una columna con tipo, tamaño y regla de nulidad.
- Cada identificador pasó a ser una llave primaria INT GENERATED ALWAYS AS IDENTITY, con la restricción nombrada.
- Los atributos únicos (el correo institucional y los nombres de los catálogos) quedaron con restricciones UNIQUE.
- Las relaciones 1:N se resolvieron con una llave foránea en el lado “muchos”.
- La relación 1:1 entre estudiante y perfil social se resolvió con una FK que además es UNIQUE.
- Las relaciones N:M quedaron como tablas intermedias, con llave primaria compuesta por las dos llaves foráneas.
- Las relaciones reflexivas (seguir y enviar mensajes) tienen dos FK hacia estudiante, una por cada rol.
- Los atributos con valores fijos (estado, rol, nivel, tipo de reacción) se controlan con CHECK.

- Las reglas de integridad RI-01 a RI-09 se implementaron con NOT NULL, llaves foráneas, UNIQUE y CHECK.

  Al pasar a físico tuvimos que ajustar algunas cosas que el modelo lógico no resolvía:

1. **Fechas con hora.** En el modelo lógico las fechas de publicación, comentario, reacción, recurso, valoración y mensaje eran DATE. Como en una red social importa la hora exacta, las pasamos a timestamptz. De hecho, fecha_envio del mensaje se describía como “fecha y hora” pero estaba definida como DATE.
2. **Restricciones de dominio nuevas.** Agregamos CHECK que antes no estaban escritos: semestre entre 1 y 10, creditos mayor que cero y los valores permitidos de nivel, estado, rol, asistencia, tipo de reacción y puntuación.
3. **Personas distintas.** Una sesión de mentoría exige mentor distinto de aprendiz, nadie puede seguirse a sí mismo y nadie puede enviarse mensajes a sí mismo.
4. **Llave foránea compuesta en las sesiones.** El modelo lógico tenía tres FK simples en sesion_mentoria: mentor, aprendiz y materia. En el físico, mentor y materia forman una FK compuesta hacia mentor_materia_habilitada. Así la base de datos garantiza que solo se agende una sesión con un mentor que esté habilitado en esa materia. La materia sigue relacionada, pero a través de la habilitación.
5. **Índices.** Creamos índices sobre las llaves foráneas que no quedan cubiertas por una PK compuesta o por un UNIQUE.

Reglas de nomenclatura y estándares

Aplicamos las reglas del libro Estructura, reglas y estándares de la unidad:

Elemento	Regla	Ejemplo
Tablas	Minúsculas, snake_case, singular y en español	estudiante, sesion_mentoria
Columnas	Minúsculas, snake_case, sin tildes	fecha_publicacion
Llaves	Prefijo codigo_ y nombre de la entidad	codigo_estudiante
FK con rol	El rol va en el nombre	codigo_mentor, codigo_seguido
Catálogos	Prefijo tipo_ cuando clasifican un atributo	tipo_grupo
Llave primaria	pk_<tabla>	pk_estudiante
Llave foránea	fk_<tabla>_<referencia>	fk_publicacion_estudiante
Único	uq_<tabla>_<campo>	uq_estudiante_correo
Check	chk_<tabla>_<campo>	chk_valoracion_rango
Índice	idx_<tabla>_<campo>	idx_publicacion_estudiante

En los diagramas y el informe de la Tarea 2 los nombres aparecían en mayúsculas solo por legibilidad. PostgreSQL convierte a minúsculas los identificadores sin comillas, así que en el modelo físico usamos minúsculas y evitamos las comillas dobles. Los valores de los CHECK también van sin tilde ('tecnico', 'asistio') para evitar errores de escritura al insertar datos.

Inventario de tablas

Módulo	Tablas
1. Usuarios y datos académicos	programa_academico, materia_asignatura, estudiante, perfil_social
2. Mentorías	mentor_materia_habilitada, sesion_mentoria
3. Perfiles y preferencias	interes, habilidad, estudiante_interes, estudiante_habilidad
4. Publicaciones e interacciones	tipo_publicacion, publicacion, seguidor_estudiante, comentario, reaccion
5. Recursos académicos	tipo_recurso, recurso_academico, valoracion_recurso
6. Comunidades y eventos	tipo_grupo, grupo, miembro_grupo, evento, asistencia_evento
7. Comunicación interna	mensaje

Son 24 tablas en total.

Implementación de las relaciones y acciones referenciales

Criterios que aplicamos

Usamos ON DELETE CASCADE cuando el registro hijo no tiene sentido sin su padre, como un perfil sin estudiante, una reacción sin publicación o cualquier fila de una tabla intermedia. Usamos ON DELETE RESTRICT cuando borrar el padre destruiría información que debe conservarse o que otros usan: los catálogos en uso, los creadores de grupos y eventos, y el historial de sesiones de mentoría.

ON UPDATE CASCADE lo aplicamos en todas las llaves foráneas como política uniforme y defensiva, para que un cambio de llave se propague. Como las PK son IDENTITY y no se espera que cambien, en la práctica esta acción casi nunca se ejecuta.

Tabla de relaciones

La tabla padre es la referenciada y la hija es la que contiene la llave foránea.

#	Padre	Hija	FK en la hija	ON DELETE	ON UPDATE	Justificación
1	programa_academico	materia_asignatura	codigo_programa	RESTRICT	CASCADE	No se borra un programa con materias
2	programa_academico	estudiante	codigo_programa	RESTRICT	CASCADE	No se borra un programa con estudiantes (RI-05)
3	estudiante	perfil_social	codigo_estudiante (UQ)	CASCADE	CASCADE	El perfil no existe sin el estudiante (RI-04)

4	estudiante	mentor_materia_habilitada	codigo_estudiante	CASCADE	CASCADE	La habilitación pierde sentido sin el estudiante
5	materia_asignatura	mentor_materia_habilitada	codigo_materia	CASCADE	CASCADE	La habilitación pierde sentido sin la materia
6	mentor_materia_habilitada	sesion_mentoria	(codigo_mentor, codigo_materia)	RESTRICT	CASCADE	Se conserva el historial; no se borra una habilitación con sesiones (RI-07)
7	estudiante	sesion_mentoria	codigo_aprendiz	RESTRICT	CASCADE	No se borra al aprendiz con sesiones (RI-07)
8	estudiante	estudiante_interes	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia
9	interes	estudiante_interes	codigo_interes	CASCADE	CASCADE	Tabla intermedia
10	estudiante	estudiante_habilidad	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia
11	habilidad	estudiante_habilidad	codigo_habilidad	CASCADE	CASCADE	Tabla intermedia
12	estudiante	publicacion	codigo_estudiante	CASCADE	CASCADE	Si se elimina al autor se

						eliminan sus publicaciones (RI-01)
13	tipo_publicacion	publicacion	codigo_tipo_publicacion	RESTRICT	CASCADE	No se borra un tipo en uso
14	estudiante	seguidor_estudiante	codigo_seguidor	CASCADE	CASCADE	El seguimiento depende de ambos estudiantes
15	estudiante	seguidor_estudiante	codigo_seguido	CASCADE	CASCADE	Ídem
16	estudiante	comentario	codigo_estudiante	CASCADE	CASCADE	Se eliminan los comentarios del autor
17	publicacion	comentario	codigo_publicacion	CASCADE	CASCADE	Un comentario no existe sin su publicación
18	estudiante	reaccion	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia
19	publicacion	reaccion	codigo_publicacion	CASCADE	CASCADE	Tabla intermedia
20	estudiante	recurso_academico	codigo_estudiante	CASCADE	CASCADE	Se eliminan los recursos del autor (RI-08)

21	materia_asignatura	recurso_academico	codigo_materia	RESTRICT	CASCADE	No se borra una materia con recursos (RI-08)
22	tipo_recurso	recurso_academico	codigo_tipo_recurso	RESTRICT	CASCADE	No se borra un tipo en uso
23	estudiante	valoracion_recurso	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia
24	recurso_academico	valoracion_recurso	codigo_recurso	CASCADE	CASCADE	Una valoración no existe sin el recurso
25	estudiante	grupo	codigo_estudiante_creador	RESTRICT	CASCADE	No se deja un grupo sin creador (RI-02)
26	tipo_grupo	grupo	codigo_tipo_grupo	RESTRICT	CASCADE	No se borra un tipo en uso
27	estudiante	evento	codigo_estudiante_creador	RESTRICT	CASCADE	No se deja un evento sin creador (RI-03)
28	estudiante	miembro_grupo	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia
29	grupo	miembro_grupo	codigo_grupo	CASCADE	CASCADE	Tabla intermedia
30	estudiante	asistencia_evento	codigo_estudiante	CASCADE	CASCADE	Tabla intermedia

31	evento	asistencia_ evento	codigo_evento	CASCADE	CASCADE	Tabla intermedia
32	estudiante	mensaje	codigo_emisor	CASCADE	CASCADE	Se eliminan los mensajes enviados (RI-09)
33	estudiante	mensaje	codigo_receptor	CASCADE	CASCADE	Se eliminan los mensajes recibidos (RI-09)

Las reglas de nomenclatura se mantuvieron iguales en los tres motores: minúsculas, snake_case, prefijos pk_, fk_, uq_, chk_ e idx_. El script de PostgreSQL fue la base y los otros dos son traducciones.

PostgreSQL 16

Creamos la base de datos redsocial_pascualina desde pgAdmin 4.

1. **Ejecutamos** ddl_red_social_pascualina_postgresql.sql desde la herramienta *Query Tool*. El script borra las tablas existentes (drop table if exists ... cascade) y las crea en orden de dependencia.
2. **Características propias de este motor** que usa el script:
    - Claves con integer generated always as identity.
    - Fechas de interacción con timestamptz (fecha y hora con zona horaria).
    - Booleano real (boolean).

- Campo metadatos_dispositivo de tipo jsonb, con la restricción chk_publicacion_metadatos (jsonb_typeof(...) = 'object').
- Comentarios con comment on table y comment on column.
- Índice GIN idx_publicacion_metadatos (jsonb_path_ops) para buscar dentro del JSON.

3. **Verificamos** con:

sql

```
select count(*) from information_schema.tables where table_schema='public';

select count(*) from information_schema.table_constraints

 where table_schema='public' and constraint_type='FOREIGN KEY';
```

Resultado: **24 tablas, 33 llaves foráneas, 10 UNIQUE, 15 CHECK y 25 índices** (24 B-tree y 1 GIN).

Inventario de las 24 tablas creadas en PostgreSQL.

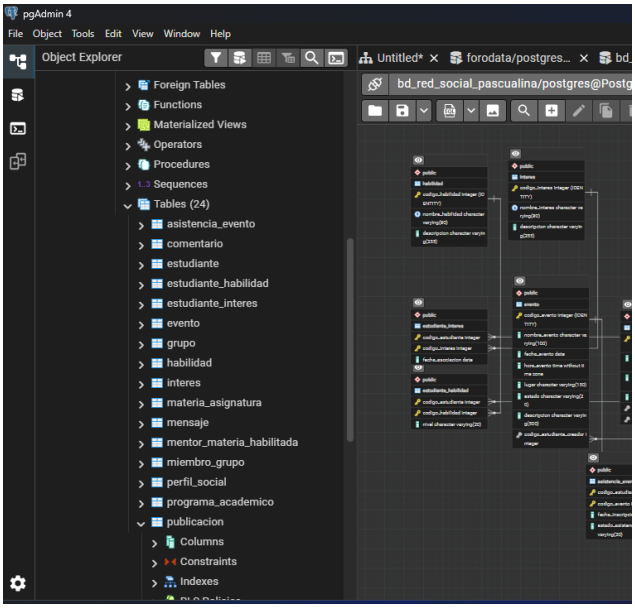
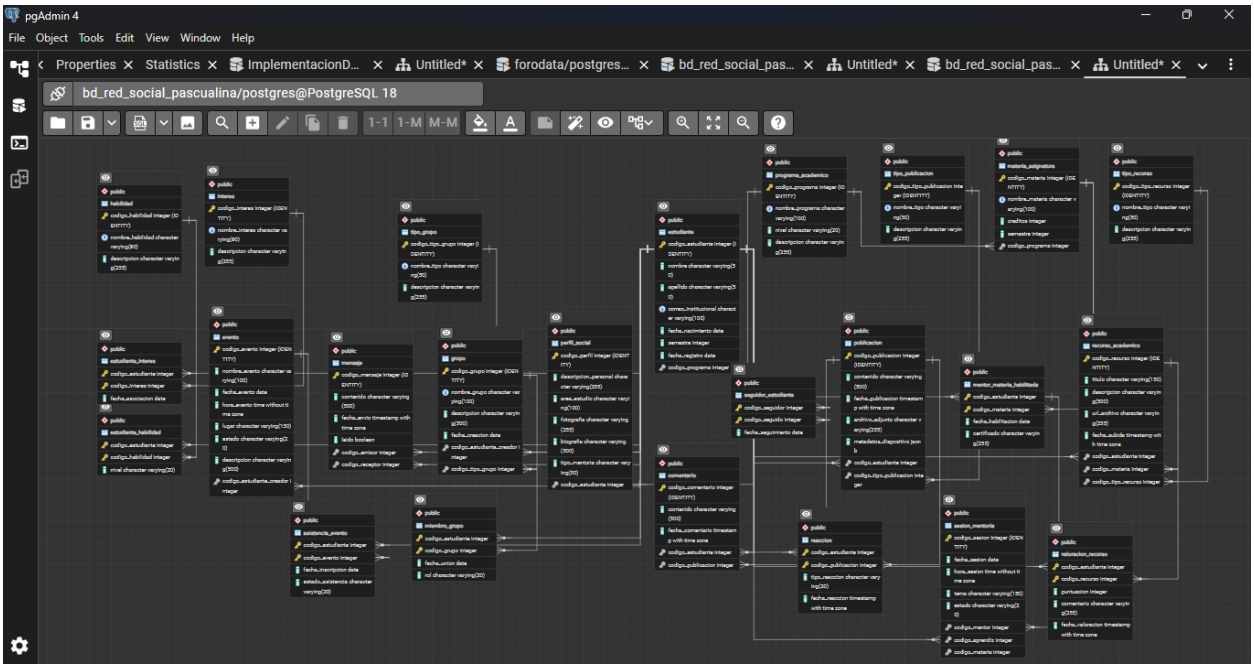


Diagrama de las 24 tablas creadas en PostgreSQL.



MySQL 8.0+

1. **Creamos la base de datos:**

sql

create database redsocial_pascualina character set utf8mb4;

use redsocial_pascualina;

2. **Ejecutamos** ddl_red_social_pascualina_mysql.sql en MySQL Workbench.

3. **Traducimos desde PostgreSQL** estos elementos:

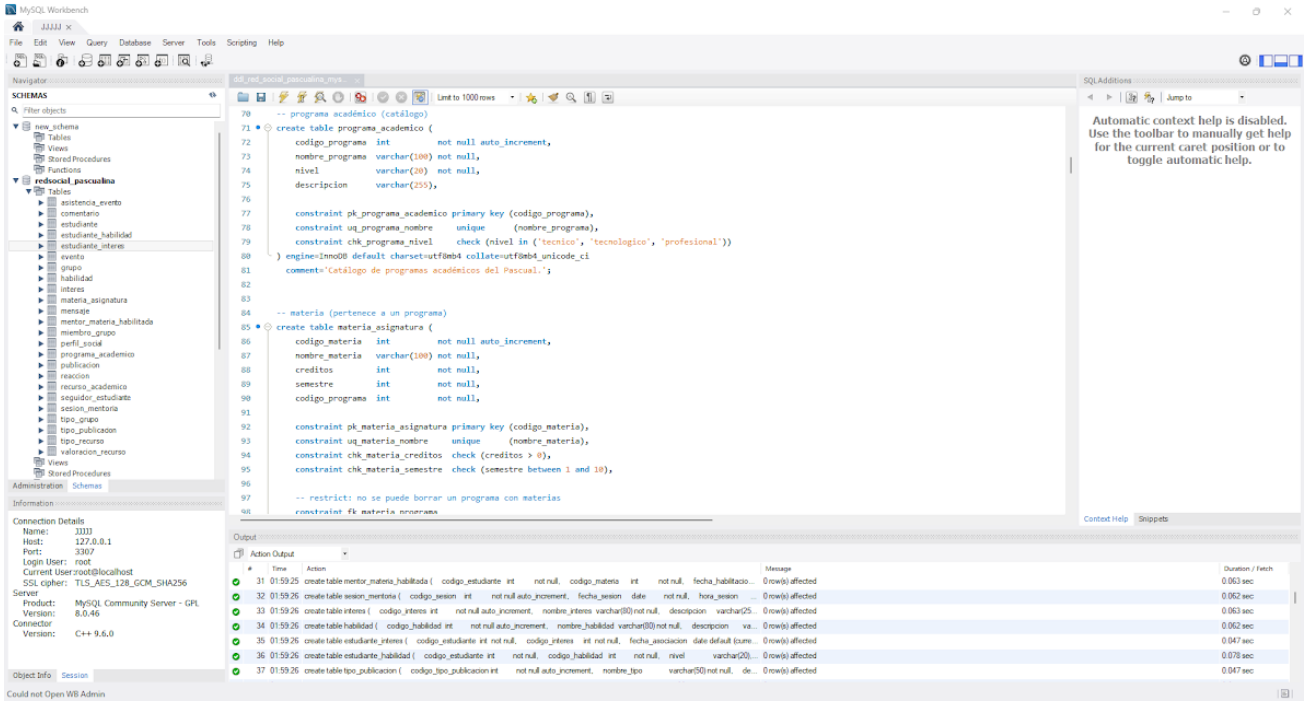
    - generated always as identity pasó a auto_increment.

    - timestamptz pasó a datetime, porque MySQL no tiene un tipo con zona horaria. La recomendación es guardar siempre en UTC desde la aplicación.

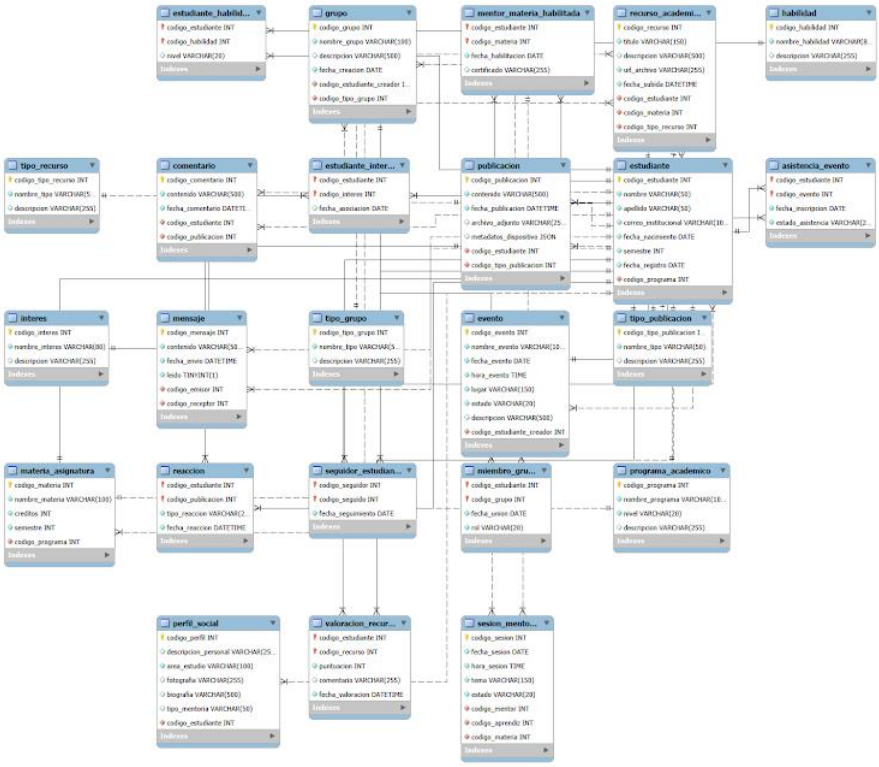
- boolean queda como tinyint(1).
- jsonb pasó a json. MySQL lo guarda en formato binario, parecido a jsonb, pero no tiene índice GIN. La validación de objeto usa json_type(...) = 'OBJECT'.
- Los valores por defecto de fecha llevan paréntesis: default (current_date).
- comment on pasó a la cláusula comment dentro de cada create table.
- Se quitó cascade del drop table y se respetó el orden de borrado.
- Cada tabla declara engine=InnoDB y utf8mb4.

4. **Ajustamos 6 llaves foráneas** a ON UPDATE RESTRICT (en sesion_mentoria, seguidor_estudiante y mensaje), porque MySQL rechaza un CHECK sobre una columna que tiene ON UPDATE CASCADE (error 3823). Se conservó ON DELETE CASCADE donde correspondía.
5. **Verificamos** con las mismas consultas, cambiando table_schema por 'redsocial_pascualina'. Resultado: **24 tablas, 33 llaves foráneas, 10 UNIQUE, 15 CHECK y 24 índices**.
6. **Probamos el JSON:** la inserción válida se guardó; el JSON mal formado fue rechazado (error 3140) y el arreglo [1,2,3] también (error 3819, restricción chk_publicacion_metadatos).
7. **Índice sobre el JSON.** Como no hay GIN, el script deja comentado un índice funcional sobre una clave del JSON, para crearlo solo si hace falta.

Inventario de las 24 tablas creadas en MySQL.



Vista gráfica de las 24 tablas implementadas en MySQL



MariaDB

1. **Creamos la base de datos:**

sql

create database redsocial_pascualina character set utf8mb4;

use redsocial_pascualina;

2. **Ejecutamos** ddl_red_social_pascualina_mariadb.sql en HeidiSQL. Mantiene la misma estructura que el de MySQL.

3. **Traducimos desde PostgreSQL** estos elementos:

    - generated always as identity pasó a auto_increment.

    - timestamptz pasó a datetime, por la misma razón que en MySQL.

- boolean queda como tinyint(1).
- jsonb pasó a json. **Aquí está la diferencia con MySQL:** en MariaDB json es un alias de longtext con validación automática (json_valid), es decir, se guarda como texto y no en formato binario. No existen jsonb ni índice GIN.
- La validación de objeto usa json_type(...) = 'OBJECT'.
- Los valores por defecto de fecha llevan paréntesis: default (current_date).
- comment on pasó a la cláusula comment en cada tabla y columna.
- Se quitó cascade del drop table.
- Cada tabla declara engine=InnoDB y utf8mb4.

4. **Ajustamos las mismas 6 llaves foráneas** a ON UPDATE RESTRICT. MariaDB también rechaza un CHECK sobre una columna con ON UPDATE CASCADE, pero con otro número de error: **1901**.
5. **Verificamos** con las mismas consultas (table_schema = ' redsocial_pascualina'). Resultado: **24 tablas, 33 llaves foráneas, 10 UNIQUE, 24 índices y 16 CHECK**. Son 15 más una adicional que el propio motor crea para validar el JSON (json_valid).
6. **Probamos el JSON:** la inserción válida se guardó; el JSON mal formado y el arreglo [1,2,3] fueron rechazados con el error 4025.
7. **Índice sobre el JSON.** MariaDB no tiene GIN, así que se indexa una columna virtual que extrae la clave consultada:

sql

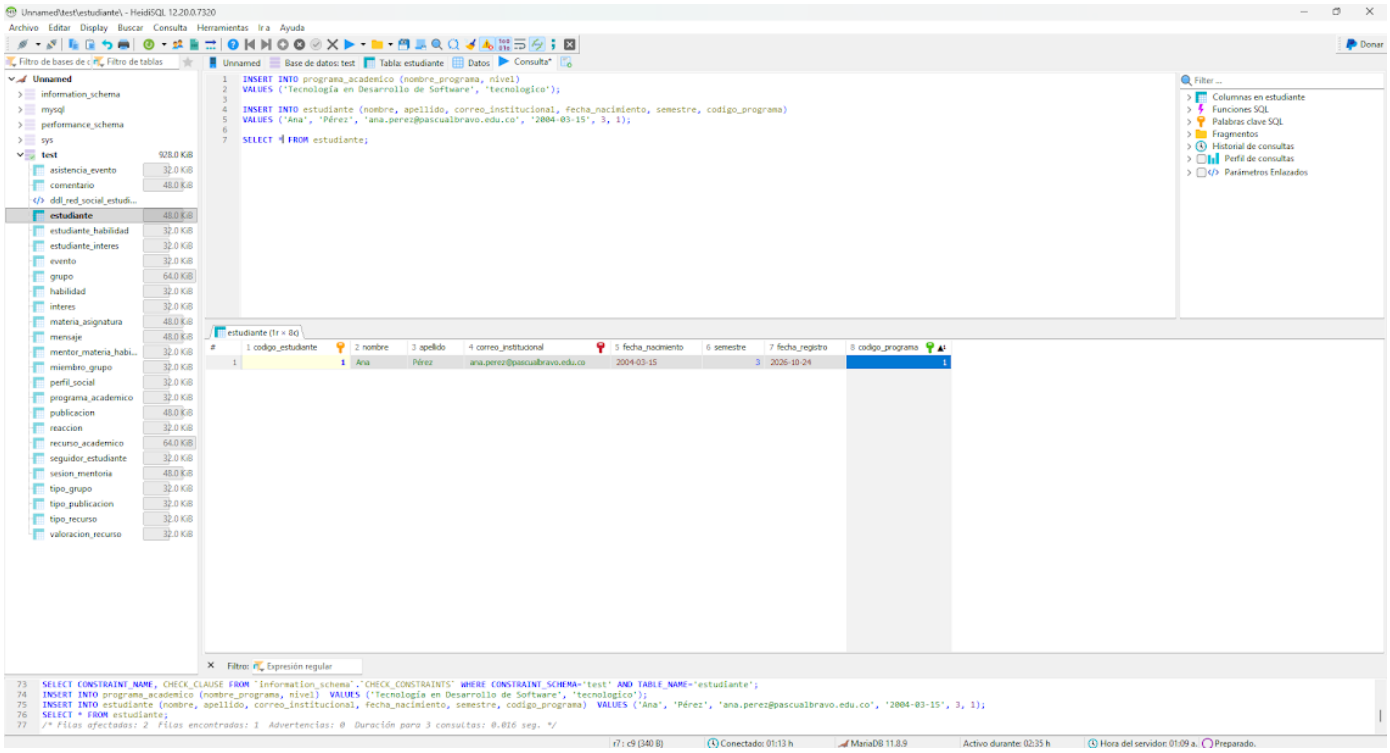
```
alter table publicacion
  add column so_dispositivo varchar(50)
```

as (json_value(metadatos_dispositivo, '$.sistema_operativo')) virtual,

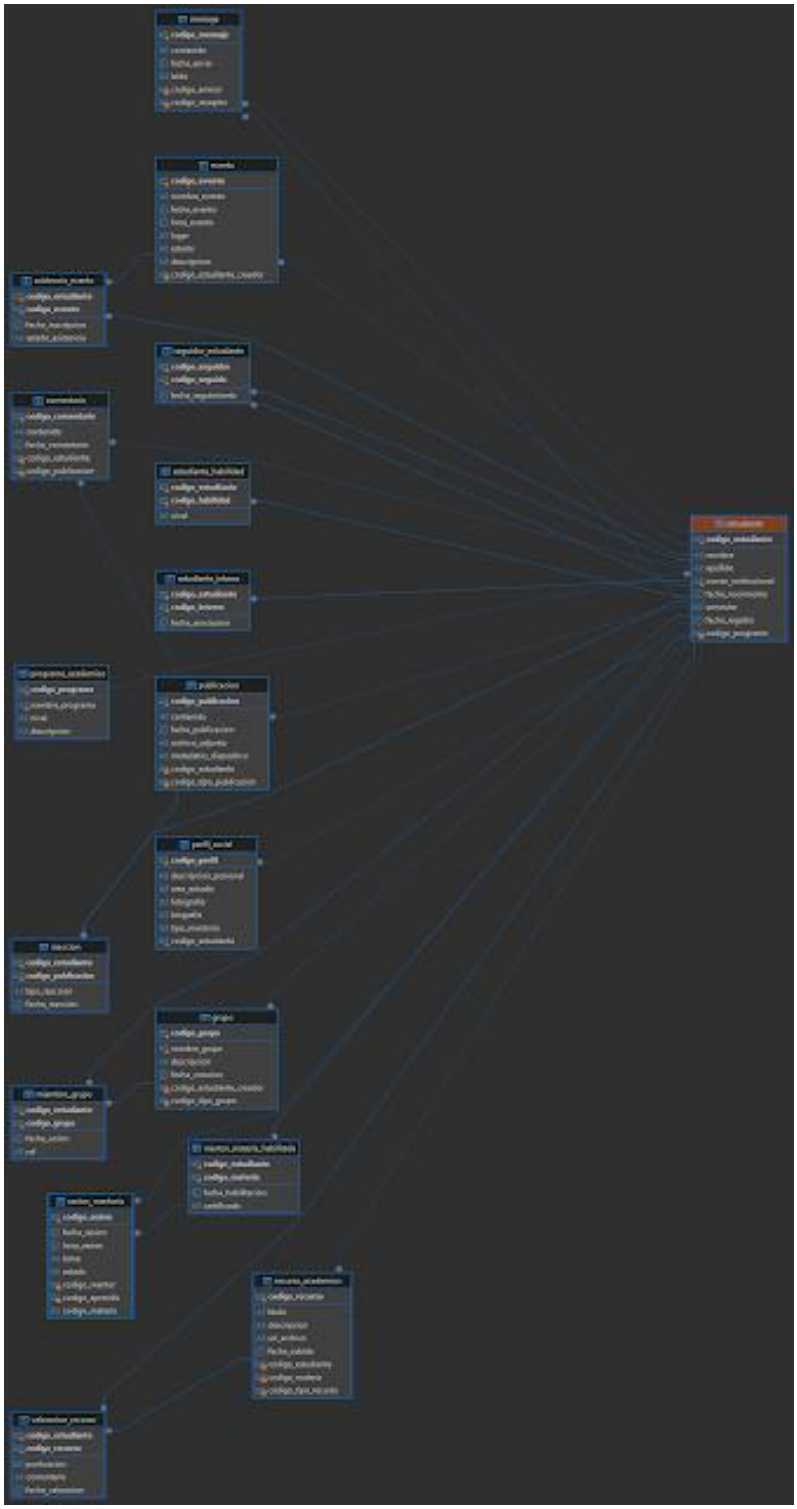
add index idx_publicacion_so (so_dispositivo);

Al consultar por so_dispositivo, el plan de ejecución (EXPLAIN) mostró que usa idx_publicacion_so. Esta parte queda comentada en el script.

Inventario de las 24 tablas creadas en MariaDB.



Vista gráfica de las 24 tablas implementadas en MariaDB



Tipos de dato SQL y NoSQL para big data e IoT

El campo metadatos_dispositivo

Agregamos a publicacion un campo JSON opcional con la telemetría del dispositivo desde el que se publica:

```
json

{

  "navegador": "Mozilla Firefox v131.0",

  "sistema_operativo": "Android 14 (One UI 6.1)",

  "tipo_conexion": "Wi-Fi Institucional (Pascual_Estudiantes)",

  "resolucion_pantalla": "1080x2340 píxeles",

  "huella_dispositivo": "fp_pascual_9876543210_xyz",

  "id_aplicacion": "pascualina-app-v3.2.0",

  "latencia_ms": 42

}
```

Por qué un dato semiestructurado

- **La forma cambia.** Cada navegador, sistema o versión de la app reporta datos distintos. Con columnas fijas habría que modificar la tabla (ALTER TABLE) cada vez que aparezca un dato nuevo.
- **Evita columnas vacías.** Un dato que solo reporta un tipo de dispositivo dejaría nulos en millones de filas.

- **Se lee junto a la publicación.** Los metadatos no se relacionan con otras tablas; son información descriptiva de ese registro.
- **Mantiene la integridad donde importa.** Autores, tipos y relaciones siguen en columnas relacionales con llaves y restricciones. Solo lo variable va en JSON. Es un enfoque híbrido SQL/NoSQL, el mismo tema de las lecturas de la unidad sobre almacenamiento híbrido, sin necesidad de montar un motor NoSQL aparte.

Relación con big data e IoT

En una red social, cada publicación genera un objeto de telemetría. A escala hay mucho **volumen** (millones de objetos), **velocidad** (inserciones constantes) y **variedad** (estructuras distintas por dispositivo): las tres características típicas de big data.

Con IoT el paralelo es directo: el celular actúa como un dispositivo que reporta lecturas (latencia, tipo de conexión, resolución), igual que un sensor que envía mediciones con formato propio. Estos datos permiten análisis como la latencia promedio por sistema operativo o el uso por tipo de red.

	PostgreSQL	**MySQL**	**MariaDB**
Tipo	jsonb	json	longtext (alias json)
Almacenamiento	Binario	Binario	Texto
Validación de objeto	jsonb_typeof(...) = 'object'	json_type(...) = 'OBJECT'	json_type + json_valid automático

Índice	GIN jsonb_path_ops (creado)	Índice funcional	Columna virtual + índice
Consulta de un campo	metadatos_dispositivo->>'sistema_operativo'	json_unquote(json_extract(metadatos_dispositivo,'$.sistema_operativo'))	La misma que MySQL

Pruebas realizadas

Probamos en los tres motores: una inserción válida con el ejemplo anterior, un JSON mal formado y un arreglo [1,2,3] (que no es un objeto).

Caso	PostgreSQL	MySQL	MariaDB
Objeto válido	Se guarda	Se guarda	Se guarda
JSON mal formado	Error de sintaxis JSON	Error 3140	Error 4025
Arreglo [1,2,3]	Viola chk_publicacion_metadatos	Error 3819	Error 4025

Las consultas por campo devolvieron el mismo resultado en los tres: sistema operativo “Android 14 (One UI 6.1)” y latencia 42.

Consideración de privacidad

El campo huella_dispositivo puede identificar un aparato y, por tanto, a una persona, así que se considera dato personal. Conforme a la Ley 1581 de 2012, debe guardarse como identificador anónimo (hash), con consentimiento del usuario y recolectando solo lo necesario.

Comparación del diccionario físico

El diccionario físico completo de cada motor está en la carpeta diccionarios/. Esta tabla resume las diferencias de tipos y de sintaxis:

Concepto	PostgreSQL 16	MySQL 8.0	MariaDB 10.11
Clave autogenerada	int generated always as identity	int auto_increment	Int(11) auto_increment
Fecha y hora	timestamptz	datetime	datetime
Fecha	date	date	date
Booleano	boolean	tinyint(1)	tinyint(1)
JSON	jsonb	json	longtext
Texto	varchar(n)	varchar(n) utf8mb4	varchar(n) utf8mb4
Comentarios	comment on	comment en la tabla	comment en la tabla
Borrado de tablas	drop table ... cascade	drop table en orden	drop table en orden
ON UPDATE en 6 FK	CASCADE	RESTRICT	RESTRICT
Restricciones CHECK	15	15	16
Índices idx_	25 (incluye GIN)	24	24

Las 6 llaves foráneas con ON UPDATE RESTRICT son las de sesion_mentoria, seguidor_estudiante y mensaje. La razón se explica en la sección *Dificultades, ventajas y desventajas*.

Dificultades, ventajas y desventajas

Dificultad principal: CHECK con ON UPDATE CASCADE

En esas tres tablas hay un CHECK que impide que las dos personas sean la misma (por ejemplo, que un estudiante se siga a sí mismo). Las columnas de ese CHECK también son llaves foráneas con ON UPDATE CASCADE.

Lo probamos con una tabla mínima:

- **MySQL** rechaza la creación con el error **3823**: la columna no puede usarse en un CHECK porque la necesita una acción referencial de una llave foránea.
- **MariaDB** la rechaza con el error **1901**.
- **PostgreSQL** la acepta sin problema.

Solución: en MySQL y MariaDB cambiamos esas 6 llaves a ON UPDATE RESTRICT. No pierde funcionalidad real, porque los códigos los genera el motor y nunca se modifican. Conservamos ON DELETE CASCADE donde correspondía.

Otras dificultades de la traducción

- La **zona horaria** se pierde en MySQL y MariaDB: datetime guarda la hora tal cual. Elegimos datetime y no TIMESTAMP porque este último se detiene en 2038. La recomendación es guardar siempre en UTC desde la aplicación.
- **Valores por defecto de fecha:** hay que escribirlos con paréntesis, default (current_date).
- **Borrado de tablas:** sin CASCADE, el orden del drop debe respetar las llaves foráneas.
- **Indexar el JSON:** no hay índice GIN; el índice hay que construirlo según el motor (sección 5.4).

Ventajas y desventajas

Motor	Ventajas	Desventajas
PostgreSQL	jsonb indexable con GIN; timestamptz; boolean real; comment on; acepta CHECK con ON UPDATE CASCADE; drop cascade	Mayor configuración inicial y más exigente en recursos que los otros dos
MySQL	Muy difundido; json binario validado; índices funcionales; Workbench gratuito	Sin zona horaria en fechas; error 3823; boolean es tinyint(1); comentarios mezclados con la definición de la tabla
MariaDB	Código abierto y compatible con MySQL; validación JSON automática; columnas virtuales indexables	El JSON es texto (se interpreta en cada consulta); misma limitación 1901; errores menos descriptivos (4025 genérico)

Conclusiones grupales

1. El **modelo lógico no depende del motor**: las mismas 24 tablas, 33 llaves foráneas y 10 UNIQUE se implementaron en los tres. Lo que cambia es el tipo de dato, la sintaxis y las restricciones del motor.
2. **No todo es portable.** Una restricción válida en PostgreSQL (CHECK con ON UPDATE CASCADE) falla en MySQL y MariaDB; hay que probar los scripts en cada motor y no asumir que funcionan.
3. **MySQL y MariaDB parecen iguales y no lo son.** El JSON lo demuestra: binario en MySQL, texto en MariaDB.
4. Un modelo **híbrido** es viable: columnas relacionales para lo que necesita integridad y JSON para lo variable, sin sumar otro motor.
5. Probar con errores intencionales (JSON mal formado, arreglo) mostró en qué punto actúa cada validación.

Propuestas de mejora

- Agregar un tipo espacial (POINT) a evento para guardar la ubicación y mejorar las búsquedas por lugar.
- Crear una tabla de lecturas de sensores por evento (aforo, temperatura), con un campo JSON para los datos variables.
- Particionar publicacion por fecha si el volumen crece.
- En MySQL y MariaDB, crear los índices sobre las claves del JSON que más se consulten.
- Sacar a columnas propias las claves del JSON que se usen mucho

Conclusiones individuales

Karen Julieth Castro Arias

Me tocó el módulo de publicaciones e interacciones, y lo más difícil fue entender por qué el mismo campo JSON se comporta distinto en cada motor. En PostgreSQL lo pude indexar con GIN y consultar por clave sin problema. En MySQL el campo es binario y la consulta funciona parecido, pero el índice hay que construirlo de otra forma. En MariaDB me sorprendió que el JSON sea texto y que el motor lo valide al insertar de todas formas. Aprendí que el modelo lógico no cambia, pero el rendimiento y la forma de consultar sí. Como mejora, en MariaDB crearíamos una columna virtual indexada sobre sistema_operativo para no escanear toda la tabla cuando crezca.

Jorge Eliecer Mazo Uparela

A mí me tocó sobre todo comparar PostgreSQL y MySQL. Lo que más me sorprendió fue el error 3823 de MySQL: no deja poner un CHECK sobre una columna que tiene on update cascade. Tuvimos que cambiar seis llaves foráneas a on update restrict en MySQL y MariaDB. Al principio pensé que habíamos roto la integridad, pero después entendí que como los códigos los genera el motor, en la práctica esas columnas nunca cambian. Aprendí que portar un script no es copiar y pegar; hay que probar en cada motor. Y también que PostgreSQL es más permisivo con el estándar, aunque eso no siempre es una ventaja si después toca migrar.

Samuel Hincapié Pérez

Mi parte fue revisar los diccionarios y el campo JSON. La dificultad más grande fue decidir cómo justificar el campo sin que pareciera un capricho. Terminamos usando el ejemplo de la telemetría del dispositivo: navegador, sistema operativo, tipo de conexión, latencia. Eso es un caso claro de big data, porque son millones de publicaciones con estructuras ligeramente distintas. En IoT el paralelo es directo: el celular actúa como un sensor que reporta latencia y

resolución. Aprendí que un campo JSON no reemplaza el modelo relacional; lo complementa. Los datos que necesitan integridad siguen en columnas con llaves. Como mejora, agregaría una tabla de lecturas de sensores por evento, con su propio JSON.

Ali Jose Arias Basto

Trabajé el módulo de recursos académicos y la traducción a MariaDB. Lo más difícil fue pelear con las diferencias que parecen menores: default (current_date) con paréntesis, drop table sin cascade, engine=InnoDB en cada tabla. Son cosas que en PostgreSQL se dan por sentadas y en MariaDB hay que escribir. Me sorprendió que MariaDB y MySQL se parezcan tanto en sintaxis y difieran tanto en cómo guardan el JSON: uno binario, otro texto. Eso me hizo entender que "compatible" no significa "idéntico". Aprendí a leer los mensajes de error del motor, porque el 1901 y el 4025 no dicen mucho y toca interpretarlos. Como mejora, probaría los scripts con datos masivos para medir la diferencia real entre JSON binario y JSON texto.

Paula Andrea Morales Zapata

Mi parte fue el módulo de comunidades y eventos, y también revisar las relaciones y las acciones referenciales en los tres motores. Lo que más me costó fue entender por qué las seis llaves foráneas de seguidor_estudiante, mensaje y sesion_mentoria tenían que cambiar a on update restrict en MySQL y MariaDB. Al principio me pareció una pérdida, pero cuando probamos el script y vimos que el motor los acepta, entendí que la integridad real la da el CHECK, no el on update. Aprendí que las decisiones de diseño físico dependen del motor, aunque el modelo lógico sea el mismo. Como mejora, en MySQL y MariaDB sacaría a columnas propias las claves del JSON que más se consulten, para no depender de un índice funcional que puede volverse costoso.